

03장 tkinter 심화 (tkinter Advanced)

tkinter 심화 개요

앞 장에서 tkinter의 기본 위젯과 이벤트 처리 흐름을 익혔다면, 이번 장에서는 GUI 프로그램을 더 체계적으로 구성하는 방법을 살펴봅니다.

간단한 프로그램은 함수 중심의 절차적 코드만으로도 충분히 만들 수 있습니다. 하지만 버튼 하나, 입력창 하나가 늘어날 때마다 전역 변수가 많아지고, 위젯과 함수가 서로 얽히면서 코드를 읽고 고치기가 점점 어려워집니다. 기능이 많아질수록 "동작하는 코드"를 넘어 "정리된 코드"를 고민해야 하는 이유입니다.

이 장은 그 고민을 단계적으로 풀어 갑니다. 먼저 절차적 코드가 어디에서 한계를 드러내는지 확인하고, 이를 객체 지향 프로그래밍(OOP)으로 다시 구성하면 어떤 점이 나아지는지 비교해 봅니다. 화면을 클래스로 묶어 설계하는 방법을 익힌 뒤, 기존에 작성했던 프로그램을 실제로 리팩토링하면서 구조가 어떻게 바뀌는지 체감하게 됩니다.

이어서 Model(데이터와 비즈니스 로직), View(화면표시와 사용자 입력받기), Presenter (Model과 View를 연결하는 중재자) 로 프로그램의 코드를 역할에 따라 나누는 아키텍처를 가볍게 맛봅니다. 처음부터 완벽한 MVP 구조를 만들기보다는, 역할을 분리한다는 사고방식 자체에 익숙해지는 데 초점을 둡니다.

마지막으로는 시간이 오래 걸리는 작업을 실행할 때 화면이 멈추는 문제를 다룹니다. GUI와 스레드를 함께 사용해 작업을 백그라운드로 넘기고, 화면이 응답성을 유지하도록 만드는 기본 원리를 알아봅니다.

이 과정을 거치면 tkinter 프로그램을 단순히 동작하게 만드는 수준을 넘어, 유지보수하기 쉽고 확장 가능한 구조로 작성하는 기초를 갖추게 됩니다. 여기서 익힌 설계 감각은 tkinter뿐 아니라 PySide6 기반의 더 큰 규모의 프로그램을 다룰 때에도 그대로 이어집니다.

목차

- [03-01 데이터 상태와 바인딩](#)
- [03-02 절차적 코드의 한계](#)
- [03-03 OOP 기반 GUI 구조 설계](#)
- [03-04 MVP 패턴](#)
- [03-05 GUI와 스레드](#)
- [03-06 큐\(Queue\)](#)

데이터 상태와 바인딩

GUI 프로그램을 작성하다 보면 한 번쯤 다음과 같은 고민을 하게 됩니다.

- "입력창에 적힌 값을 어떻게 가져오지?"
- "버튼을 누르면 화면의 글자를 어떻게 바꾸지?"

단순한 구조에서는 위젯을 직접 제어하여 대응할 수 있지만, UI가 복잡해질수록 데이터의 흐름과 상태 변화를 수동으로 추적하는 것은 유지보수를 크게 저하시킵니다.

이 장에서는 이러한 복잡성을 해소할 핵심 개념으로 Tkinter 변수 클래스와 데이터 바인딩(Data Binding)을 다룹니다. 이를 적용하면 위젯을 직접 조작하는 방식이 아니라 데이터의 상태 변화만으로 화면이 자동 동기화되는 효율적인 프로그램을 작성할 수 있습니다.

- [상태\(State\)와 데이터 바인딩](#)
 - [상태\(State\)란?](#)
 - [데이터 바인딩이란?](#)
- [tkinter 변수 클래스](#)
 - [4가지 변수 클래스](#)
 - [값을 넣고 읽는 방법](#)
- [코드로 따라 하기](#)
 - [라벨과 변수 묶기](#)
 - [변수의 값 변경 시 라벨도 변경](#)
 - [라디오버튼](#)

상태(State)와 데이터 바인딩

- 두 가지 핵심 용어부터 짚고 넘어가겠습니다.

상태(State)란?

- 상태란 한마디로 '프로그램이 지금 이 순간 기억하고 있는 데이터'를 말합니다. 거창해 보이지만, 우리가 화면에서 보는 거의 모든 것이 상태입니다.
 - 체크박스에 체크가 되어 있는가, 아닌가
 - 사용자가 입력창에 어떤 글자를 적었는가
 - 지금 화면에 보이는 숫자가 몇인가
- 이런 것들이 모두 프로그램의 '상태'입니다. 프로그램을 만든다는 것은 결국 이 상태를 잘 보관하고, 잘 바꾸고, 화면에 잘 보여 주는 일 이라고 할 수 있습니다.

데이터 바인딩이란?

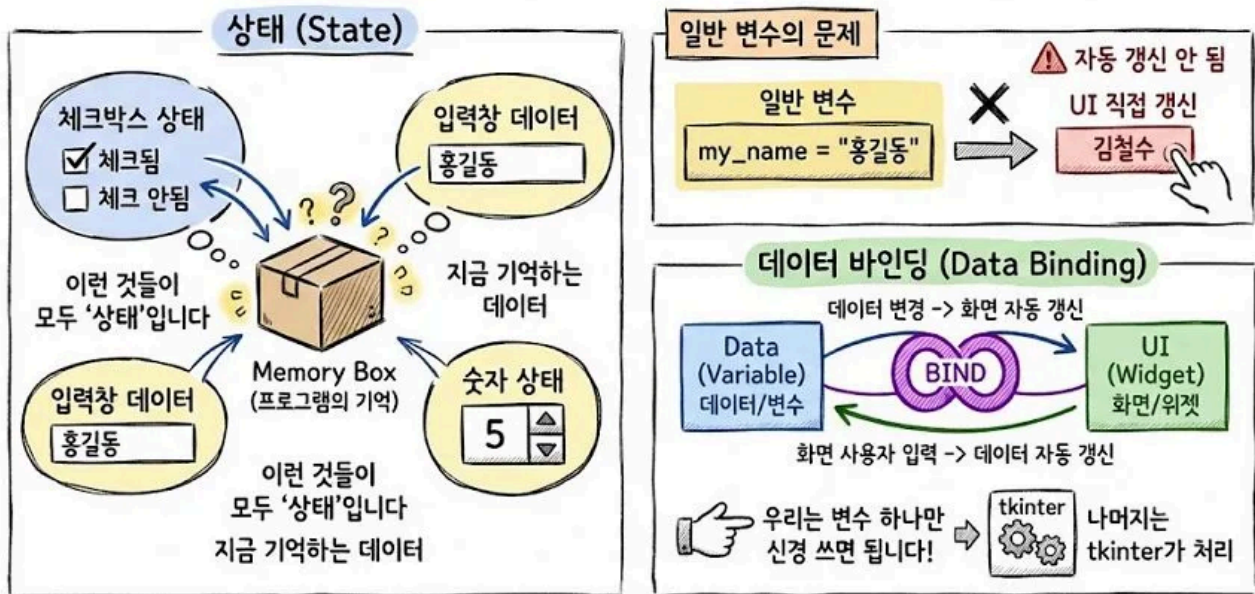
- 문제는, 우리가 평소에 쓰던 일반 변수로는 상태를 화면과 연결하기가 번거롭다는 점입니다.

```
my_name = "홍길동"    # 일반 변수
```

- 이렇게 만든 변수는 값이 바뀌어도 화면에 그 사실을 스스로 알려 주지 않습니다. 화면을 새 값으로 바꾸고 싶다면, 우리가 코드로 매번 직접 라벨이나 텍스트를 갱신해 주어야 합니다.

비유하자면, 일반 변수는 "내 값이 바뀌었어!"라고 아무에게도 말하지 않는 조용한 변수입니다.

- 데이터 바인딩은 이 불편함을 없애 줍니다. 데이터 바인딩이란 '데이터(변수)'와 '화면(위젯)'을 하나로 묶어(Bind) 버리는 것 입니다. 한 번 묶어 두면 다음이 자동으로 일어납니다.
 - 변수의 값을 바꾸면 → 화면이 알아서 바뀐다
 - 화면에서 사용자가 값을 바꾸면 → 변수 안의 데이터도 알아서 바뀐다
- 즉, 우리는 화면이 아니라 변수 하나만 신경 쓰면 됩니다. 나머지는 tkinter가 대신 처리해 줍니다.



tkinter 변수 클래스

- 데이터 바인딩을 하려면, 파이썬의 일반 변수 대신 tkinter 전용 변수 클래스를 써야 합니다.
- 이 변수들은 값이 바뀔 때마다 화면에 "값이 바뀌었어!"라고 신호를 보내 주는 특별한 변수입니다.

4가지 변수 클래스

- 데이터의 종류에 따라 4가지 변수 클래스가 있습니다.

변수 클래스	담는 값	주로 쓰는 곳
<code>tk.StringVar()</code>	문자열(글자)	입력창, 레이블 텍스트
<code>tk.IntVar()</code>	정수(숫자)	라디오버튼, 숫자 값
<code>tk.DoubleVar()</code>	실수(소수점이 있는 숫자)	슬라이더, 진행률
<code>tk.BooleanVar()</code>	참/거짓(True/False)	체크박스 선택 여부

값을 넣고 읽는 방법

- 이 변수 클래스들은 일반 파이썬 변수처럼 `=` 기호로 값을 넣을 수 없습니다.
- 대신 `.set()` 과 `.get()` 이라는 전용 메서드를 사용합니다.

```

my_var = tk.StringVar()          # 먼저 변수 클래스로 만든다

my_var.set("변수 클래스")         # 값 넣기
current_value = my_var.get()     # 값 읽기

```

앞으로 나오는 모든 예제가 이 두 메서드를 사용합니다.

코드로 따라 하기

- 이제 개념을 실제 코드로 하나씩 확인해 보겠습니다.

라벨과 변수 묶기

- 위젯과 변수를 묶을 때는 위젯을 만들 때 `textvariable` 이나 `variable` 옵션에 변수를 넣어 주면 됩니다.
- 다음은 레이블과 변수를 묶는 예제입니다.

```

import tkinter as tk

root = tk.Tk()

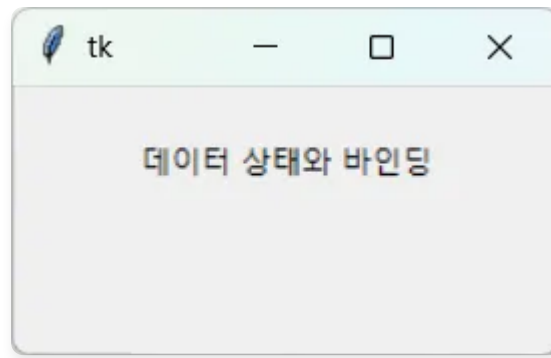
message = tk.StringVar()
message.set("데이터 상태와 바인딩")

label = tk.Label(root, textvariable=message)
label.pack(padx=20, pady=20)

root.mainloop()

```

- 핵심은 `textvariable=message` 부분입니다. `text` 대신 `textvariable` 에 변수를 넣었기 때문에, 이제 레이블은 `message` 변수와 한 몸이 되었습니다.



📌 알아두기

라벨에는 `textvariable`, 라디오버튼·체크박스에는 `variable` 옵션을 사용합니다. 이름은 다르지만 "변수를 묶는다"는 역할은 같습니다.

변수의 값 변경 시 라벨도 변경

- 정말로 변수와 위젯이 연결되어 있는지 직접 확인해 봅시다.
- 버튼을 누르면 변수의 값만 바꾸는데도 화면의 라벨이 따라서 바뀌는 예제입니다.

```
import tkinter as tk

root = tk.Tk()

floor = tk.IntVar()
floor.set(1)

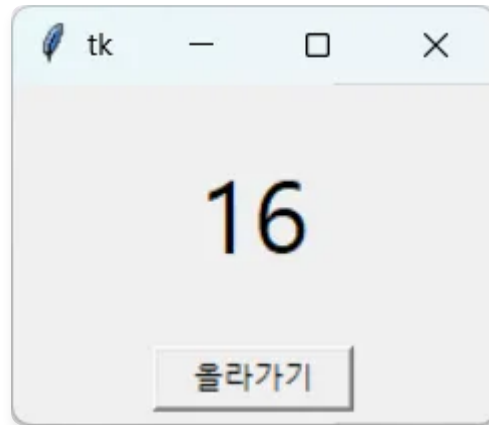
label = tk.Label(root, textvariable=floor, font=("맑은 고딕", 30))
label.pack(padx=30, pady=20)

def go_up():
    floor.set(floor.get() + 1)

button_up = tk.Button(root, text="올라가기", command=go_up, width=10)
button_up.pack(pady=5)

root.mainloop()
```

- 이 예제에서 `go_up()` 함수는 `label` 에 대해 아무 명령도 내리지 않습니다. 오직 `floor.set(...)` 으로 변수 값만 바꿀 뿐입니다.
- 그런데도 버튼을 누를 때마다 화면의 숫자가 1씩 올라갑니다.
- 변수와 레이블이 묶여 있기 때문에 변수가 바뀌면 화면이 알아서 따라오는 것입니다. 이것이 데이터 바인딩의 힘입니다.



라디오버튼

- 여러 선택지 중 하나만 고르는 라디오버튼은 `IntVar` 로 관리합니다.
- 각 라디오버튼에 `value` 값을 정해 주고, 같은 변수에 묶어 두면 어떤 것을 골랐는지 변수 하나로 알 수 있습니다.


```

import tkinter as tk

def show_result():
    label.config(text="난이도 : " + level_var.get())

root = tk.Tk()
level_var = tk.StringVar(value="쉬움")

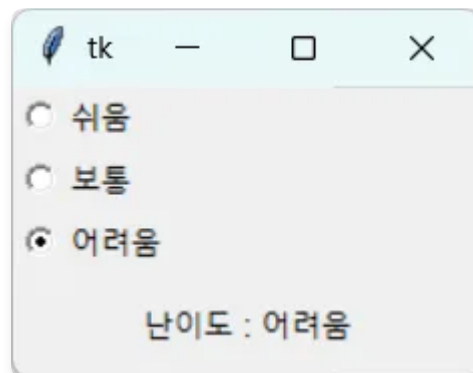
for level in ["쉬움", "보통", "어려움"]:
    tk.Radiobutton(root, text=level, variable=level_var, value=level,
                    command=show_result).pack(anchor="w")

label = tk.Label(root, text="")
label.pack(pady=10)

root.mainloop()

```

- 세 개의 라디오버튼이 모두 `level_var` 라는 하나의 변수에 묶여 있기 때문에, 사용자가 어떤 것을 고르더라도 `level_var.get()` 한 줄로 선택 결과를 알 수 있습니다.



절차적 코드의 한계

이번 장에서는 간단한 "할 일 목록" 프로그램을 절차적 방식으로 만들어 봅니다. 처음에는 짧고 직관적이던 코드가 기능이 하나씩 늘어날 때마다 어떻게 복잡해지는지 단계별로 따라가 보겠습니다.

이 과정을 직접 겪고 나면, 다음 장에서 배울 "OOP기반 GUI 구조 설계"가 왜 필요한지 자연스럽게 이해할 수 있습니다.

- [들어가며](#)
 - [이 장의 목표](#)
 - [무엇을 만들 것인가](#)
- [단계별로 만들어 보기](#)
 - [최소 기능](#)
 - [코드 한 줄씩 살펴보기](#)
 - [삭제 기능 추가](#)
 - [코드 한 줄씩 살펴보기](#)
 - [현실적인 요구사항 추가](#)
 - [코드 한 줄씩 살펴보기](#)
- [절차적 코드의 4가지 한계](#)
 - [global 키워드의 남발](#)
 - [위젯과 변수가 같은 공간에 노출](#)
 - [같은 화면을 여러 개 만들 수 없음](#)
 - [상태와 동작의 분리](#)
 - [정리하며](#)

들어가며

이 장의 목표

- 절차적 방식으로만 프로그램 하나를 단계적으로 만들어 봅니다.
- 기능이 하나씩 늘어날 때마다 코드가 어떻게 부풀고, 어디서부터 손대기 어려워지는지 직접 따라가 봅니다.

- 이 과정을 겪고 나면, 다음 장의 "OOP 기반 GUI 구조 설계"가 왜 필요한지 자연스럽게 이해하게 됩니다.

무엇을 만들 것인가

- tkinter로 만드는 간단한 "할 일 목록(To-Do List)" 프로그램입니다.
- 한 번에 완성하지 않고, 세 단계에 걸쳐 기능을 점점 늘려 갑니다.
- 최소 기능 → 삭제 기능 → 완료 처리·상태 표시·전체 비우기 순입니다.
- 핵심은 완성된 결과물이 아니라 늘려 가는 과정입니다.
- 단계가 올라갈수록 전역 변수가 늘고, global 선언이 곳곳에 붙고, 관련 코드가 멀리 흩어집니다.
- 바로 그 불편함이 이 장의 주제이며, "절차적 코드의 네 가지 한계"로 이어집니다.

단계별로 만들어 보기

최소 기능

- 먼저 '할 일'을 입력하고 추가하면 목록에 올라가는, 가장 단순한 코드입니다.

```

import tkinter as tk

root = tk.Tk()
root.geometry("300x250")

listbox = tk.Listbox(root)
listbox.pack(fill="both", expand=True, padx=10, pady=(0, 7))

entry = tk.Entry(root)
entry.pack(fill="x", padx=10, pady=10)

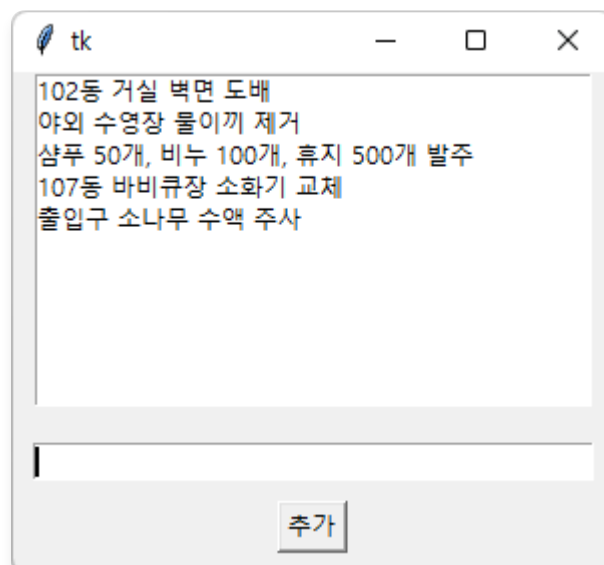
def add_item():
    text = entry.get().strip()
    if text:
        listbox.insert(tk.END, text)
        entry.delete(0, tk.END)

button = tk.Button(root, text="추가", command=add_item)
button.pack(pady=(0, 10))

root.mainloop()

```

이 정도 규모에서는 절차적 코드도 문제없이 잘 돌아갑니다. 오히려 클래스보다 짧고 직관적으로 보이니까도 합니다.



코드 한 줄씩 살펴보기

- `entry` 값 가져오기

```
def add_item():
    text = entry.get().strip()
    if text:
        listbox.insert(tk.END, text)
        entry.delete(0, tk.END)
```

- `entry.get()` 은 입력칸에 적힌 글자를 가져오고, `.strip()` 은 앞뒤 공백을 잘라냅니다.
- `insert(tk.END, text)` 의 `tk.END` 는 "목록 맨 끝"을 뜻합니다. 새 항목이 항상 마지막에 붙습니다.

삭제 기능 추가

- 이번에는 선택한 항목을 지우는 기능을 더합니다.

```

import tkinter as tk

root = tk.Tk()
root.geometry("300x250")

listbox = tk.Listbox(root)
listbox.pack(fill="both", expand=True, padx=10, pady=(0, 7))

entry = tk.Entry(root)
entry.pack(fill="x", padx=10, pady=10)

def add_item():
    text = entry.get().strip()
    if text:
        listbox.insert(tk.END, text)
        entry.delete(0, tk.END)

def delete_item():
    selected_indices = listbox.curselection()
    if selected_indices:
        listbox.delete(selected_indices[0])

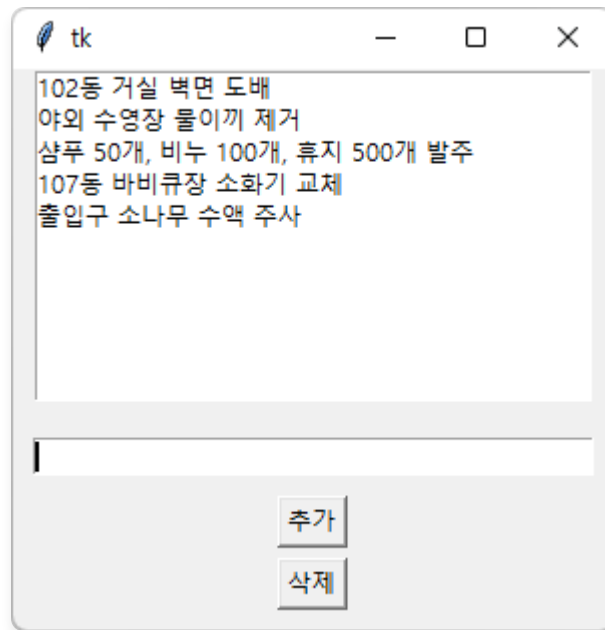
add_button = tk.Button(root, text="추가", command=add_item)
add_button.pack(pady=(0, 5))

delete_button = tk.Button(root, text="삭제", command=delete_item)
delete_button.pack(pady=(0, 10))

root.mainloop()

```

약간의 기능이 추가되었지만 짧고 단순한 프로그램은 절차적 코드를 사용하는 것도 괜찮습니다.



코드 한 줄씩 살펴보기

- `listbox`의 선택 값 삭제하기

```
def delete_item():
    selected_indices = listbox.curselection()
    if selected_indices:
        listbox.delete(selected_indices[0])
```

- `curselection()`은 사용자가 고른 항목의 위치를 튜플로 돌려줍니다.
- `selected_indices[0]`은 첫 번째로 선택된 항목의 위치입니다. 그 위치의 항목을 `delete`로 지웁니다.

현실적인 요구사항 추가

- 이제 실제 프로그램에 가까운 기능을 더해 봅니다.
- 완료 처리, 전체 항목 수와 완료 수를 보여주는 상태 라벨, 전체 비우기입니다.

```

import tkinter as tk

root = tk.Tk()
root.geometry("300x400")

total_count = 0
done_count = 0

listbox = tk.Listbox(root)
listbox.pack(fill="both", expand=True, padx=10, pady=(0, 7))

entry = tk.Entry(root)
entry.pack(fill="x", padx=10, pady=10)

status_label = tk.Label(root, text="전체 0개 / 완료 0개")
status_label.pack()

def update_status():
    global total_count, done_count
    status_label.config(text=f"전체 {total_count}개 / 완료 {done_count}개")

def add_item():
    global total_count
    text = entry.get().strip()
    if text:
        listbox.insert(tk.END, text)
        entry.delete(0, tk.END)
        total_count += 1
        update_status()

def delete_item():
    global total_count
    selected = listbox.curselection()
    if selected:
        listbox.delete(selected[0])
        total_count -= 1
        update_status()

def mark_done():
    global done_count

```



```

selected = listbox.curselection()
if selected:
    text = listbox.get(selected[0])
    listbox.delete(selected[0])
    listbox.insert(selected[0], "[완료] " + text)
    done_count += 1
    update_status()

def clear_all():
    global total_count, done_count
    listbox.delete(0, tk.END)
    total_count = 0
    done_count = 0
    update_status()

add_button = tk.Button(root, text="추가", command=add_item)
add_button.pack()

done_button = tk.Button(root, text="완료 처리", command=mark_done)
done_button.pack()

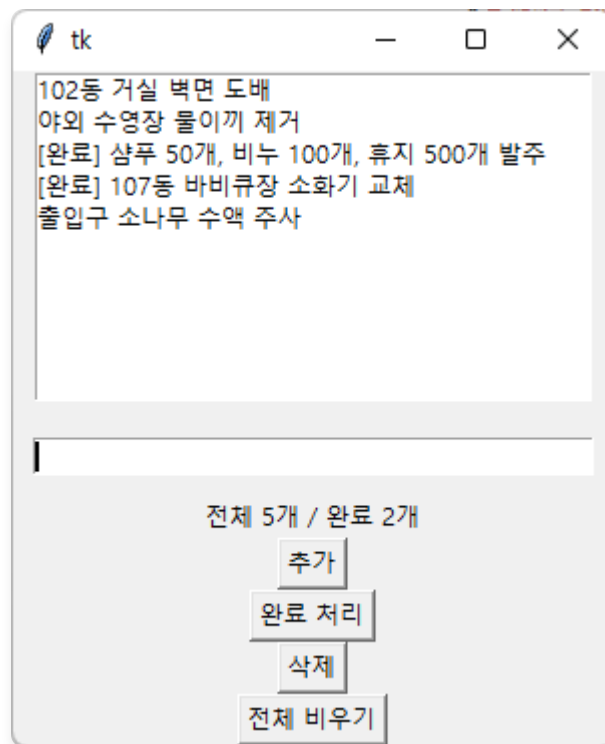
delete_button = tk.Button(root, text="삭제", command=delete_item)
delete_button.pack()

clear_button = tk.Button(root, text="전체 비우기", command=clear_all)
clear_button.pack()

root.mainloop()

```

요구사항 증가에 따라 코드의 스파게티화가 진행되어 버렸습니다.



코드 한 줄씩 살펴보기

- 전역 변수 생성

```
total_count = 0
done_count = 0
```

함수 바깥(전역)에 만든 변수입니다. 전체 항목 수와 완료 수를 프로그램 전체에서 공유하려고 여기에 둡니다.

- 전역 변수 가져오기

```
def update_status():
    global total_count, done_count
    status_label.config(text=f"전체 {total_count}개 / 완료 {done_count}개")
```

- `global total_count, done_count` 는 "함수 안에서 쓰는 이름이 새 지역 변수가 아니라 위에서 만든 전역 변수다"라고 알려 주는 선언입니다.
- `status_label.config(text=...)` 는 이미 만들어진 라벨의 글자를 새로 바꾸는 명령입니다.

절차적 코드의 4가지 한계

global 키워드의 남발

- 상태 변수 `total_count`, `done_count`가 전역이라, 이 값을 바꾸는 모든 함수가 `global` 선언을 해야 합니다.
- 위 코드에 `global` 이 다섯 번이나 등장하고, 함수가 늘수록 계속 따라붙습니다.
- `global` 이 많아질수록 어떤 함수가 어떤 상태를 건드리는지 파악하기 어렵습니다.

```
def mark_done():
    # global done_count ← 이 줄을 빼뜨리면?
    done_count += 1      # 에러는 없지만, 카운터가 증가하지 않습니다.
```

- `global` 을 깜빡하면 파이썬은 조용히 새 지역 변수를 만들어, 값이 반영되지 않는 버그가 생깁니다.
- 에러 메시지 없이 동작만 이상해지므로 찾기가 까다롭습니다.

위젯과 변수가 같은 공간에 노출

- `entry`, `listbox`, `status_label`, 버튼 다섯 개까지 모든 위젯이 전역 공간에 놓여 있습니다.
- 함수도 상태 변수도 같은 공간이라, 프로그램이 커지면 수십 개의 이름으로 가득 찹니다.
- 이름이 많아지면 충돌 위험도 커집니다. 무심코 `entry` 를 다시 쓰면 기존 것이 덮어써져 기능이 망가집니다.
- 어떤 위젯이 어느 기능에 속하는지 코드 구조만으로는 알 수 없습니다.

같은 화면을 여러 개 만들 수 없음

- 가장 본질적인 한계입니다. 위 코드는 "할 일 목록 하나"에 완전히 묶여 있습니다.
- 한 창에 개인용·업무용 목록을 나란히 두고 싶어도 방법이 없습니다. `listbox`, `total_count`, `add_item`이 하나씩만 존재하기 때문입니다.

- 두 번째 목록을 만들려면 `listbox2`, `total_count2`처럼 전부 복사해 이름만 바꿔야 하고, 세 개면 또한 벌을 복사합니다.
- 반면 `CounterFrame` 같은 클래스는 `CounterFrame(root)`를 두 번 호출하면 독립된 인스턴스가 둘 생기고, 각자의 상태와 위젯을 가져 서로 간섭하지 않습니다.

상태와 동작의 분리

- `total_count` 데이터와 이를 바꾸는 `add_item`, `delete_item`, `clear_all`은 논리적으로 한 덩어리입니다.
- 하지만 절차적 코드에서는 변수는 위, 함수는 가운데, 위젯은 아래로 흩어져 있습니다.
- 관련 코드가 떨어져 있으면 기능 하나를 고칠 때 곳곳을 오가야 합니다.
- "할 일 목록 로직"만 떼어 재사용하려 해도 경계가 분명하지 않아 추출이 어렵습니다.

정리하며

한계	핵심 증상
<code>global</code> 남발	상태 변경 함수마다 선언이 붙고, 빠뜨리면 조용한 버그
이름 공간 오염	모든 이름이 한 공간에 뒤섞여 충돌 위험
복제 불가	같은 화면을 여러 개 만들 수 없음
상태·동작 분리	관련 코드가 흩어져 수정·재사용이 어려움

- 이 네 가지는 모두 "관련된 데이터와 동작을 하나로 묶지 못한다"는 한 가지 뿌리에서 나옵니다.

OOP 기반 GUI 구조 설계

이 장에서는 절차적 프로그램의 한계를 보완하기 위해, 클래스를 활용해 GUI를 구성할 때 자주 쓰이는 4가지 구조 패턴을 살펴봅니다.

- [클래스 기반 GUI 설계의 필요성](#)
 - [절차적 코드의 한계](#)
 - [클래스 도입에 따른 구조적 장점](#)
- [Tk 직접 상속](#)
 - [메인 윈도우 자체를 애플리케이션으로](#)
 - [코드 한 줄씩 살펴보기](#)
- [Frame 상속](#)
 - [재사용 가능한 부품으로 캡슐화](#)
 - [코드 한 줄씩 살펴보기](#)
- [컴포넌트 조합](#)
 - [영역별 부품을 메인 앱에서 조립](#)
 - [부품끼리 직접 알지 못하게 하기](#)
 - [코드 한 줄씩 살펴보기](#)
- [다중 페이지 전환](#)
 - [페이지를 겹쳐 두고 바꿔 올리기](#)
 - [컨트롤러가 전환을 맡는다](#)
 - [코드 한 줄씩 살펴보기](#)
- [OOP로 리팩토링](#)
 - [수정한 코드](#)

클래스 기반 GUI 설계의 필요성

절차적 코드의 한계

- 앞 강좌에서 절차적 구현이 가지는 구조적 한계를 살펴보았습니다. 위젯이 전역 변수로 흩어지고, 기능이 함수 곳곳에 분산되어 코드가 커질수록 관리가 어려워집니다.

클래스 도입에 따른 구조적 장점

- 위젯이 `self.label` 처럼 객체의 속성이 되어, 어느 메서드에서든 바로 접근할 수 있습니다.
- 관련 동작이 메서드로 모여, 코드가 기능 단위로 정리됩니다.
- 화면을 통째로 재사용하거나, 일부만 떼어 테스트하기 쉬워집니다.

알아두기

self란?

클래스 안에서 "자기 자신"을 가리키는 이름입니다. `self.label` 은 "이 객체가 가진 `label`"이라는 뜻으로, 객체 안 어디서든 같은 위젯을 가리킵니다.

Tk 직접 상속

메인 윈도우 자체를 애플리케이션으로

- `tk.Tk` 를 상속해 메인 윈도우를 곧 애플리케이션 클래스로 정의합니다.
- 구조가 단순해 소규모 앱에 적합하지만, 윈도우와 로직이 한 클래스에 묶여 확장성은 떨어집니다.

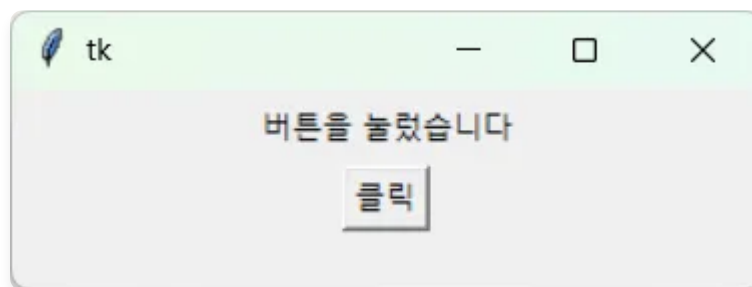
```
import tkinter as tk

class Application(tk.Tk):
    def __init__(self):
        super().__init__()
        self.geometry("300x80")
        self.label = tk.Label(self, text="안녕하세요")
        self.label.pack(pady=5)
        tk.Button(self, text="클릭", command=self.on_click).pack()

    def on_click(self):
        self.label.config(text="버튼을 눌렀습니다")

if __name__ == "__main__":
    Application().mainloop()
```

- 위젯을 `self.label` 처럼 속성으로 두었기 때문에, `on_click` 안에서 바로 접근할 수 있습니다.
- 위젯을 일일이 전역 변수로 두거나 인자로 주고받을 필요가 없습니다.



코드 한 줄씩 살펴보기

- `Tk` 초기화

```
super().__init__()
```

부모인 `Tk` 를 가장 먼저 초기화합니다. 이 줄이 빠지면 창이 제대로 만들어지지 않습니다.

Frame 상속

재사용 가능한 부품으로 캡슐화

- 실무에서는 `Tk` 를 직접 상속하기보다 `Frame` 을 상속하는 방식을 더 권장합니다.
- `Frame` 은 여러 위젯을 담는 컨테이너이므로, 이를 상속한 클래스는 그 자체로 하나의 "부품(컴포넌트)"이 됩니다. 여러 개를 조합하거나 다른 창에 끼워 넣기 쉽습니다.

```
import tkinter as tk

class CounterFrame(tk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.count = 0
        self.display = tk.Label(self, text="0", font=("맑은 고딕", 24))
        self.display.pack(pady=10)
        tk.Button(self, text="+1", command=self.increase).pack()

    def increase(self):
        self.count += 1
        self.display.config(text=str(self.count))

if __name__ == "__main__":
    root = tk.Tk()
    CounterFrame(root).pack(padx=20, pady=20)
    root.mainloop()
```




코드 한 줄씩 살펴보기

- 부모 위에 부품 엮기

```
super().__init__(master)
```

`master` (부모 위젯)에 이 `Frame` 을 소속시킵니다. 단, 실제로 화면에 나타나는 시점은 나중에 `.pack()` 을 호출할 때입니다. 즉 생성과 배치는 별개입니다.

- 숫자 표시 라벨

```
self.display = tk.Label(self, text="0", font=("맑은 고딕", 24))
```

첫 인자 `self` 는 "이 Frame 안에 넣겠다"는 뜻입니다. 부모 자리에 `self` 를 쓰는 것이 컴포넌트화의 핵심입니다.

컴포넌트 조합

영역별 부품을 메인 앱에서 조립

- 화면을 영역별(입력·표시 등)로 나누어 각각 클래스로 만들고, 메인 클래스에서 조립합니다.
- 부품 단위로 수정·테스트할 수 있어 유지보수성이 높습니다.

```

import tkinter as tk

class NameInput(tk.Frame):
    def __init__(self, master, on_submit):
        super().__init__(master)
        self.on_submit = on_submit
        self.entry = tk.Entry(self)
        self.entry.pack(side="left")
        tk.Button(self, text="인사하기", command=self.submit).pack(side="left")

    def submit(self):
        self.on_submit(self.entry.get())

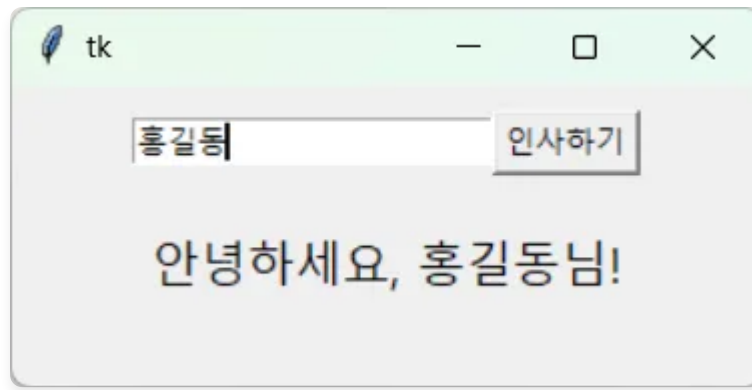
class Greeting(tk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.label = tk.Label(self, text="", font=("맑은 고딕", 14))
        self.label.pack()

    def show(self, name):
        self.label.config(text=f"안녕하세요, {name}님!")

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.geometry("300x120")
        self.greeting = Greeting(self)
        self.name_input = NameInput(self, on_submit=self.greeting.show)
        self.name_input.pack(pady=10)
        self.greeting.pack(pady=10)

if __name__ == "__main__":
    App().mainloop()

```



부품끼리 직접 알지 못하게 하기

- `NameInput` (입력)과 `Greeting` (표시)으로 나뉩니다. 핵심은 `NameInput` 이 `Greeting` 의 존재를 전혀 모른다는 점입니다.
- 버튼을 누르면 `NameInput` 은 약속된 `on_submit` 함수에 이름만 넘깁니다.
- 그 이름으로 무엇을 할지는 메인 클래스가 `on_submit=self.greeting.show` 로 연결하며 결정합니다.
- 이렇게 결합도를 낮추면, 표시 부품을 다른 것으로 바꿔도 입력 부품은 손댈 필요가 없습니다.

코드 한 줄씩 살펴보기

- 부품끼리 연결하기

```
self.name_input = NameInput(self, on_submit=self.greeting.show)
```

입력 부품의 `on_submit` 자리에 표시 부품의 `show` 를 끼워 넣어, 메인 클래스가 둘을 잇는 다리 역할을 합니다. "무엇을 할지"는 부품이 아니라 메인이 정합니다.

다중 페이지 전환

페이지를 겹쳐 두고 바꿔 올리기

- 로그인 화면에서 메인 화면으로 넘어가듯 여러 페이지를 전환해야 할 때, 각 페이지를 `Frame` 으로 만들어 같은 위치에 겹쳐 둔 뒤 보여 줄 페이지만 맨 위로 끌어올리는 패턴을 자주 씁니다.

```

import tkinter as tk

class PageOne(tk.Frame):
    def __init__(self, master, controller):
        super().__init__(master)
        tk.Label(self, text="첫 번째 페이지", font=("맑은 고딕", 16)).pack(pady=20)
        tk.Button(self, text="다음으로",
                  command=lambda: controller.show_page("PageTwo")).pack()

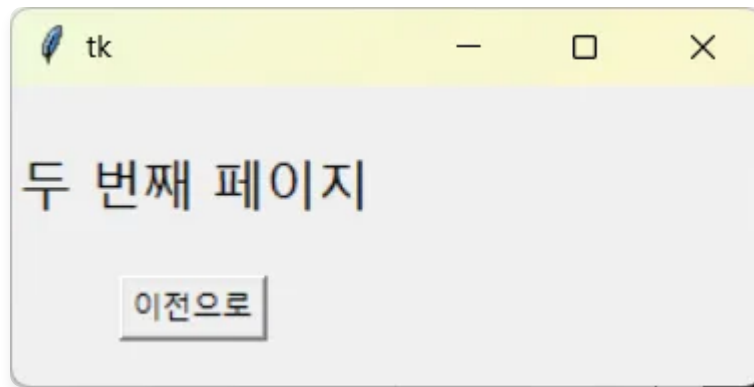
class PageTwo(tk.Frame):
    def __init__(self, master, controller):
        super().__init__(master)
        tk.Label(self, text="두 번째 페이지", font=("맑은 고딕", 16)).pack(pady=20)
        tk.Button(self, text="이전으로",
                  command=lambda: controller.show_page("PageOne")).pack()

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.geometry("300x200")
        self.pages = {}
        for PageClass in (PageOne, PageTwo):
            page = PageClass(self, self)
            self.pages[PageClass.__name__] = page
            page.grid(row=0, column=0, sticky="nsew")
        self.show_page("PageOne")

    def show_page(self, name):
        self.pages[name].tkraise()

if __name__ == "__main__":
    App().mainloop()

```



컨트롤러가 전환을 맡는다

- 모든 페이지를 `grid` 로 같은 셀(`row=0, column=0`)에 두면 서로 겹쳐 쌓입니다.
- `tkraise()` 는 지정한 페이지를 맨 위로 끌어올려, 그 페이지만 보이게 합니다.
- 메인 클래스 `App` 은 페이지들을 보관하고 전환을 담당하는 "컨트롤러" 이며, 각 페이지는 `controller` 를 통해 전환을 요청합니다.

코드 한 줄씩 살펴보기

- 페이지를 만들어 모아 두기

```
for PageClass in (PageOne, PageTwo):
    page = PageClass(self, self)
    self.pages[PageClass.__name__] = page
    page.grid(row=0, column=0, sticky="nsew")
```

페이지들을 미리 만들어 딕셔너리에 모으고, 같은 셀에 카드처럼 포개 둡니다. 두 번째 인자 `self` 가 전환을 요청할 통로(컨트롤러)이고, `sticky="nsew"` 는 셀을 가득 채우라는 뜻입니다.

- 원하는 페이지를 위로 올리기

```
def show_page(self, name):
    self.pages[name].tkraise()
```

`tkraise()` 로 지정한 페이지만 맨 위로 끌어올립니다. 겹쳐진 카드 중 하나를 골라 보여 주는 셈입니다.

- 전환 요청 보내기

```
command=lambda: controller.show_page("PageTwo")
```

버튼을 누르면 컨트롤러에게 전환을 부탁드립니다. 페이지가 직접 전환하지 않고 요청만 하는 점이 핵심입니다.

OOP로 리팩토링

수정한 코드

- 절차적 코드로 작성한 '할 일 목록' 프로그램을 클래스를 활용해 OOP 기반으로 리팩토링 해보겠습니다.

```

import tkinter as tk

class TodoApp:
    def __init__(self, root):
        self.root = root
        self.root.geometry("300x350")
        self.total_count = 0
        self.done_count = 0

        self._create_widgets()
        self.update_status()

    def _create_widgets(self):
        self.listbox = tk.Listbox(self.root)
        self.listbox.pack(fill="both", expand=True, padx=10, pady=(0, 7))

        self.entry = tk.Entry(self.root)
        self.entry.pack(fill="x", padx=10, pady=10)

        self.status_label = tk.Label(self.root)
        self.status_label.pack()

        buttons = [
            ("추가", self.add_item),
            ("완료 처리", self.mark_done),
            ("삭제", self.delete_item),
            ("전체 비우기", self.clear_all),
        ]
        for text, command in buttons:
            tk.Button(self.root, text=text, command=command).pack()

    def update_status(self):
        self.status_label.config(
            text=f"전체 {self.total_count}개 / 완료 {self.done_count}개"
        )

    def add_item(self):
        text = self.entry.get().strip()
        if text:

```

```

        self.listbox.insert(tk.END, text)
        self.entry.delete(0, tk.END)
        self.total_count += 1
        self.update_status()

def delete_item(self):
    selected = self.listbox.curselection()
    if selected:
        self.listbox.delete(selected[0])
        self.total_count -= 1
        self.update_status()

def mark_done(self):
    selected = self.listbox.curselection()
    if selected:
        index = selected[0]
        text = self.listbox.get(index)
        self.listbox.delete(index)
        self.listbox.insert(index, "[완료] " + text)
        self.done_count += 1
        self.update_status()

def clear_all(self):
    self.listbox.delete(0, tk.END)
    self.total_count = 0
    self.done_count = 0
    self.update_status()

if __name__ == "__main__":
    root = tk.Tk()
    app = TodoApp(root)
    root.mainloop()

```


MVP 패턴 적용

작은 GUI 프로그램을 만들 때 처음에는 모든 코드를 한 곳에 작성하지만 규모가 커지면 관리하기 어려워집니다. 이처럼 tkinter의 GUI 프로그램의 규모가 커지게 될 때 자주 사용되는 아키텍처가 MVP 패턴입니다.

이 장에서는 MVP를 간략히 알아보겠습니다.

- [MVP 패턴의 이해](#)
 - [MVP의 세 가지 역할](#)
 - [핵심 규칙과 데이터 흐름](#)
 - [MVP 적용 시 장점](#)
- [MVP 패턴으로 앱 만들기](#)
 - [Model](#)
 - [View](#)
 - [Presenter](#)
 - [조립과 실행](#)
 - [코드 한 줄씩 살펴보기](#)

MVP 패턴의 이해

- MVP는 **Model-View-Presenter**의 약자로, GUI 프로그램의 코드를 역할에 따라 세 부분으로 나누는 아키텍처입니다.
- 핵심 목적은 데이터, 화면, 로직을 분리하여 규모가 커져도 코드를 관리할 수 있게 만드는 것입니다.

MVP의 세 가지 역할

구성 요소	역할	tkinter에서
Model	데이터와 비즈니스 로직	순수 파이썬 클래스 (tkinter 모름)
View	화면 표시와 사용자 입력 받기	위젯 배치, 이벤트 전달
Presenter	Model과 View를 연결하는 중재자	입력 처리 후 Model 갱신, View 업데이트

핵심 규칙과 데이터 흐름

- 핵심 규칙은 "**View와 Model은 서로 직접 대화하지 않는다**" 입니다.
- 모든 흐름은 Presenter를 거칩니다.

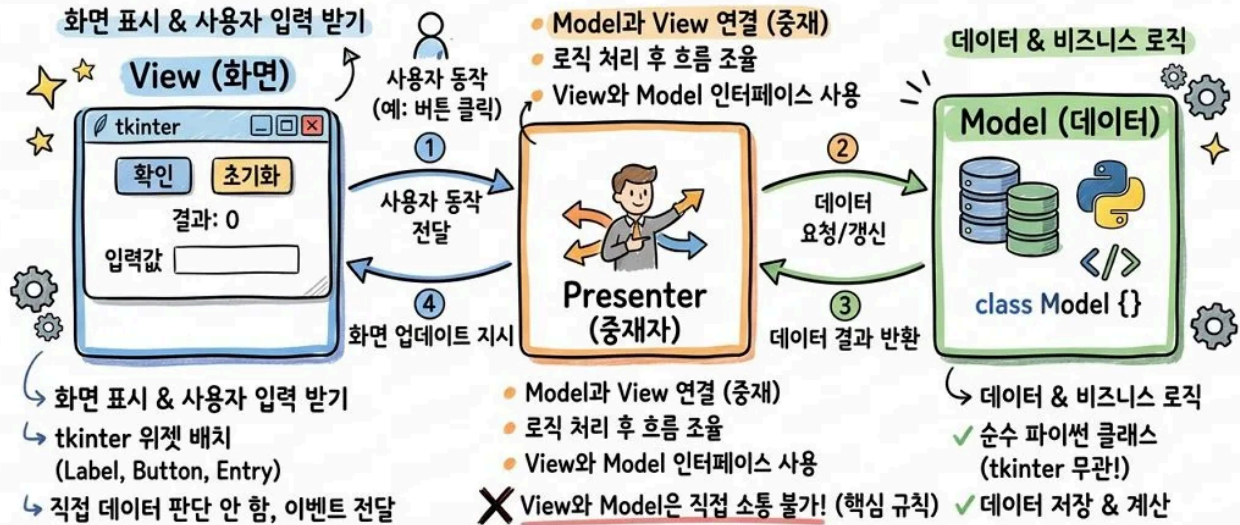
```
[사용자] → View → Presenter → Model
                ↓
[사용자] ← View ← Presenter
```

- View는 스스로 판단하지 않고 사용자 동작을 Presenter에게 "전달"만 합니다.
- Model은 tkinter를 전혀 모르는 순수한 파이썬일 뿐입니다.
- Presenter는 Model과 View를 둘 다 알고 있으며, 흐름을 조율합니다.

MVP 적용 시 장점

- 화면(View) 없이도 데이터(Model)와 로직(Presenter)을 독립적으로 단위 테스트할 수 있습니다.
- 화면을 변경해도 로직은 그대로, 로직을 수정해도 화면은 그대로 둘 수 있어 유지보수가 편리합니다.
- 디자이너는 화면에, 개발자는 로직에 집중할 수 있어 명확한 업무 분장과 협업이 가능합니다.
- 프로그램 기능이 늘어나도 역할별로 코드가 체계적으로 정리되어 확장성이 뛰어납니다.

[Model-View-Presenter]



MVP 패턴으로 앱 만들기

- 앞에서 살펴본 개념을 바탕으로 '단위 변환기 앱' 예제를 MVP로 만들어 봅니다.

Model

- Model은 `import tkinter` 가 없는 순수한 파이썬일 뿐입니다.

```
class DistanceModel:
    KM_PER_MILE = 1.609344

    def km_to_miles(self, km):
        return km / self.KM_PER_MILE

    def miles_to_km(self, miles):
        return miles * self.KM_PER_MILE
```

Model은 화면도 모르고 버튼이 있는지도 모르기 때문에 GUI 없이 단독으로 `DistanceModel` 만 테스트할 수 있습니다.

View

- View는 위젯을 만들고, 사용자 동작을 Presenter에게 "전달"만 합니다.

- 스스로 판단하지 않습니다.

```
import tkinter as tk

class DistanceView(tk.Frame):
    def __init__(self, master):
        super().__init__(master)
        self.pack(padx=15, pady=15)

        self.entry = tk.Entry(self, width=15)
        self.entry.grid(row=0, column=0, columnspan=2, pady=5)

        self.km_btn = tk.Button(self, text="km → 마일")
        self.km_btn.grid(row=1, column=0, padx=2)

        self.mile_btn = tk.Button(self, text="마일 → km")
        self.mile_btn.grid(row=1, column=1, padx=2)

        self.result = tk.Label(self, text="값을 입력하세요", width=25)
        self.result.grid(row=2, column=0, columnspan=2, pady=10)

    def get_input(self):
        return self.entry.get()

    def show_result(self, text):
        self.result.config(text=text)
```

- tkinter는 오직 View 안에만 존재합니다.
- `show_result` 처럼 Presenter가 부를 창구(메서드)만 열어두고, 그 안에서 화면을 어떻게 바꿀지는 View가 알아서 합니다.

Presenter

- Presenter는 Model과 View를 둘 다 알고 있으며, 흐름을 조율합니다.

```

class DistancePresenter:
    def __init__(self, model, view):
        self.model = model
        self.view = view

        self.view.km_btn.config(command=self.on_km_to_miles)
        self.view.mile_btn.config(command=self.on_miles_to_km)

    def _read_value(self):
        """입력값을 숫자로 변환. 실패하면 None과 함께 안내."""
        try:
            return float(self.view.get_input())
        except ValueError:
            self.view.show_result("숫자를 입력하세요")
            return None

    def on_km_to_miles(self):
        value = self._read_value()
        if value is not None:
            miles = self.model.km_to_miles(value)
            self.view.show_result(f"{value} km = {miles:.2f} 마일")

    def on_miles_to_km(self):
        value = self._read_value()
        if value is not None:
            km = self.model.miles_to_km(value)
            self.view.show_result(f"{value} 마일 = {km:.2f} km")

```

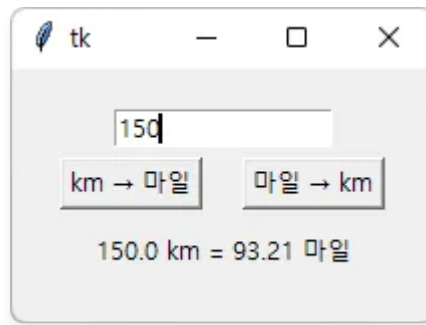
조립과 실행

- 다음 코드를 통해서 MV를 모두 조립 후 실행합니다.

```

if __name__ == "__main__":
    root = tk.Tk()
    presenter = DistancePresenter(DistanceModel(), DistanceView(root))
    root.mainloop()

```



코드 한 줄씩 살펴보기

- Presenter는 Model(계산)과 View(화면) 사이를 잇는 조정자입니다. 둘에게 일을 시키고 흐름만 조율합니다.

```
def __init__(self, model, view):
    #             ↑ 첫 번째 인자가   ↑ 두 번째 인자가
    #             model에 들어옴   view에 들어옴
    self.model = model
    self.view = view
```

- Presenter가 만들어질 때 `model` (계산 담당)과 `view` (화면 담당)를 건네받아 `self` 에 보관합니다.
- 이렇게 해두면 Presenter는 언제든지 `self.model` 로 계산을 시키고 `self.view` 로 화면을 바꿀 수 있습니다.

- `DistancePresenter(model, view)`의 `model`에 `DistanceModel()`이 들어오며, `view`에 `DistanceView(root)`가 들어옵니다.

```
DistancePresenter(DistanceModel(), DistanceView(root))
#             └─ model 자리 ─┘ └─ view 자리 ─┘
```

GUI와 멀티스레드

GUI 프로그램 개발 중 흔히 겪는 '응답 없음' 현상은 대개 하나의 작업 흐름이 모든 일을 혼자 처리하려 할 때 발생합니다.

이 강좌에서는 **스레드(Thread)** 활용해 이 문제를 해결해 봅니다. 스레드의 기본 개념과 파이썬 구현 방법을 익힌 뒤, 실제 GUI 예제를 통해 스레드 적용 전후의 차이를 직접 비교해 보겠습니다.

- [스레드 이해하기](#)
 - [스레드란](#)
 - [스레드는 왜 필요한가?](#)
- [파이썬에서 스레드 구성 방법](#)
 - [함수를 넘기는 방식](#)
 - [Thread를 상속하는 방식](#)
- ['카운트 다운' 예제 코드](#)
 - [메인 스레드에서 처리](#)
 - [무엇이 문제인가](#)
 - [작업 스레드로 분리](#)
 - [무엇이 달라졌나](#)
- [after\(\)로 안전하게 화면 갱신하기](#)
 - [위젯은 메인 스레드에서만 다루어야 함](#)
 - [그래서 필요한 것이 after\(\)](#)
 - [after\(\)만으로 카운트다운 구현하기](#)

스레드 이해하기

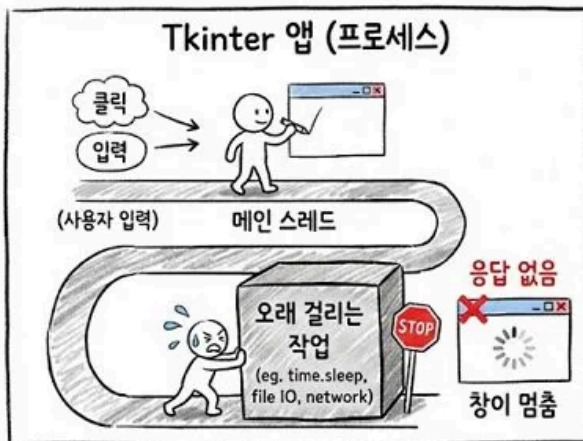
스레드란

- 프로그램을 실행하면 프로세스가 생기고, 그 안에서 실제로 코드를 실행하는 흐름이 **스레드(Thread)**입니다.
- 기본적으로 스레드는 하나(메인 스레드)지만, 필요하면 여러 개를 만들어 일을 나눌 수 있습니다.

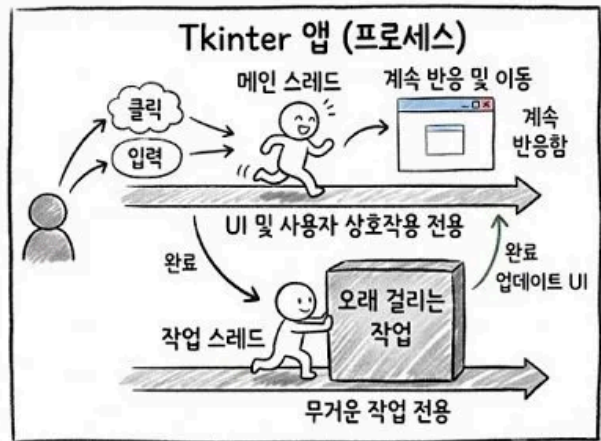
스레드는 왜 필요한가?

- 스레드가 하나면 모든 일을 순서대로 처리합니다. 그래서 오래 걸리는 작업(`time.sleep` , 파일 읽기, 네트워크 등)을 만나면 끝날 때까지 다른 일이 멈춥니다.
- GUI에서 이 문제가 바로 '응답 없음' 입니다. 메인 스레드가 대기 작업에 묶이면 창 이동·버튼 클릭에 반응하지 못합니다.
- 그래서 해결책은 역할을 나누는 것입니다.
- **메인 스레드** : 화면을 그리고 사용자 입력에 반응하는 일에 집중합니다.
- **작업 스레드** : 오래 걸리는 작업을 따로 맡아 처리합니다.
- 이렇게 하면 작업이 도는 동안에도 화면이 멈추지 않아 '응답 없음'을 피할 수 있습니다.

1. 스레드가 1개일 때 (기본)



2. 스레드가 여러 개일 때 (해결책)



핵심: 역할을 나눕니다! 메인 스레드 = 화면 & 입력 작업 스레드 = 무거운 계산

파이썬에서 스레드 구성 방법

- 파이썬은 표준 라이브러리인 `threading` 모듈로 스레드를 다룹니다. 스레드를 만드는 방식은 크게 두 가지입니다.

함수를 넘기는 방식

- 가장 간단한 방법은 `Thread` 객체를 만들면서 실행할 함수를 `target` 으로 지정하는 것입니다.


```
import threading
import time

def work():
    for i in range(5):
        print(f"작업 중 {i}")
        time.sleep(1)

t = threading.Thread(target=work)
t.start()
```

- `start()` 를 호출하면 새 스레드가 생성되어 `work` 함수를 실행합니다.
- 이때 메인 스레드는 `t.start()` 다음 줄로 곧바로 넘어가며, 두 흐름이 동시에 진행됩니다.
- 즉 `start()` 는 "이 일을 새 스레드에 맡기고 나는 계속 진행하라"는 뜻입니다.

Thread를 상속하는 방식

- 좀 더 구조적인 방법은 `threading.Thread` 를 상속한 클래스를 만들고 `run` 메서드를 재정의하는 것입니다.
- 스레드가 다룰 데이터나 상태가 많아 객체지향적으로 설계할 때 적합합니다.

```
import threading
import time

class Worker(threading.Thread):
    def __init__(self, name):
        super().__init__()
        self.worker_name = name

    def run(self):
        for i in range(5):
            print(f"{self.worker_name}: {i}")
            time.sleep(1)

w = Worker("작업자1")
w.start()
```

- 여기서 주의할 점은 `run()` 을 직접 호출하면 안 된다는 것입니다.
- `run()` 을 직접 부르면 새 스레드 없이 현재 스레드에서 그냥 메서드를 실행할 뿐입니다.
- 반드시 `start()` 를 호출해야 운영체제가 새 스레드를 만들고, 그 안에서 `run()` 을 실행해 줍니다.

📌 알아두기

`target` 에 함수를 넘기는 방식은 간단한 작업에, `Thread` 를 상속하는 방식은 상태를 가진 복잡한 작업에 어울립니다.

어느 쪽이든 실행은 항상 `start()` 로 시작한다는 원칙은 같습니다.

'카운트 다운' 예제 코드

- 이제 같은 카운트다운 프로그램을 두 가지 방식으로 만들어, 화면이 멈추는 경우와 멈추지 않는 경우를 직접 비교해 보겠습니다.
- 두 코드는 10부터 0까지 1초 간격으로 숫자를 줄여 표시하고, 마지막에 "끝!"을 보여준다는 점에서 동일합니다.
- 차이는 그 작업을 누가 처리하느냐에 있습니다.

메인 스레드에서 처리

```
import tkinter as tk
import time

class CountdownApp:
    def __init__(self, root):
        self.root = root
        self.label = tk.Label(root, text="10", font=("Arial", 60))
        self.label.pack(expand=True)
        tk.Button(root, text="시작", command=self.start).pack(pady=10)

    def start(self):
        for i in range(10, -1, -1):
            self.label.config(text=str(i))
            self.root.update_idletasks()
            time.sleep(1)
        self.label.config(text="끝!")

if __name__ == "__main__":
    root = tk.Tk()
    CountdownApp(root)
    root.mainloop()
```



무엇이 문제인가

- `time.sleep(1)` 이 메인 스레드를 붙잡아 두어 화면 갱신이나 다른 입력을 막습니다.
- 카운트다운 중 창을 움직이면 반응하지 않습니다.

- 오래 걸리는 작업을 메인 스레드에서 처리하면 결국 '응답 없음' 상태가 됨을 보여줍니다.

작업 스레드로 분리

```
import tkinter as tk
import threading
import time

class CountdownApp:
    def __init__(self, root):
        self.root = root
        self.label = tk.Label(root, text="10", font=("Arial", 60))
        self.label.pack(expand=True)
        tk.Button(root, text="시작", command=self.start).pack(pady=10)

    def start(self):
        threading.Thread(target=self.count, daemon=True).start()

    def count(self):
        for i in range(10, -1, -1):
            self.label.config(text=str(i))
            time.sleep(1)
        self.label.config(text="끝!")

if __name__ == "__main__":
    root = tk.Tk()
    CountdownApp(root)
    root.mainloop()
```

⚠ 이해를 돕기 위한 예제로 작동은 잘 되지만, **작업 스레드에서 tkinter 위젯을 직접 조작한다는 문제점**이 있습니다. 이어지는 강좌 큐(Queue)에서 조금 더 안전한 코드로 개선하는 방법을 설명합니다.



무엇이 달라졌나

- 시작 버튼을 누르면 카운트다운 작업(`count`)을 새로운 작업 스레드에 온전히 맡깁니다.
- 무거운 대기 시간이 작업 스레드에서 흐르므로 메인 스레드는 자유로운 상태가 됩니다.
- 메인 스레드가 멈추지 않아 카운트다운 중에도 창 이동, 버튼 클릭 등에 정상적으로 반응합니다.

after()로 안전하게 화면 갱신하기

위젯은 메인 스레드에서만 다루어야 함

- 앞서 작업 스레드로 분리한 코드는 화면이 멈추지 않아 잘 동작하는 것처럼 보입니다.
- 작업 스레드에서 `self.label.config(...)` 처럼 위젯을 직접 건드리면, 당장은 문제가 없어 보여도 상황에 따라 화면이 깨지거나 예기치 않게 프로그램이 멈출 수 있습니다.

그래서 필요한 것이 after()

- `after(밀리초, 함수)`는 "지정한 시간이 흐른 뒤 이 함수를 메인 스레드에서 실행하라"고 예약하는 메서드입니다.
- 즉 위젯을 바꾸는 일은 메인 스레드에 맡기고, 작업 스레드는 "언제 무엇을 할지"만 예약하는 방식으로 역할을 다시 나눌 수 있습니다.

after()만으로 카운트다운 구현하기

- 사실 카운트다운처럼 "1초마다 한 번씩"인 단순한 반복은 스레드 없이 `after()` 만으로도 깔끔하게 만들 수 있습니다.
- `time.sleep` 으로 흐름을 멈추는 대신, 다음 숫자를 1초 뒤에 실행하도록 예약하고 함수를 빠져 나오는 것이 핵심입니다.

```
import tkinter as tk

class CountdownApp:
    def __init__(self, root):
        self.root = root
        self.label = tk.Label(root, text="10", font=("Arial", 60))
        self.label.pack(expand=True)
        tk.Button(root, text="시작", command=self.start).pack(pady=10)

    def start(self):
        self.count(10)

    def count(self, i):
        if i < 0:
            self.label.config(text="끝!")
            return
        self.label.config(text=str(i))
        self.root.after(1000, self.count, i - 1)

if __name__ == "__main__":
    root = tk.Tk()
    CountdownApp(root)
    root.mainloop()
```

큐(Queue) 통신

tkinter 위젯은 메인 스레드에서만 안전하게 조작할 수 있으므로, 백그라운드 스레드가 위젯을 직접 바꾸면 간헐적으로 충돌합니다.

그래서 **큐(queue)**를 사이에 둡니다. 백그라운드 스레드는 결과를 큐에 **넣기만** 하고, 메인 스레드는 큐에서 결과를 **꺼내** 화면을 바꿉니다. 역할이 나뉘므로 위젯은 언제나 메인 스레드에서만 조작됩니다.

- [큐\(Queue\) 개념과 사용법](#)
 - [큐\(Queue\)란](#)
 - [기본 사용법](#)
- [스레드와 큐 활용하기](#)
 - ['스레드 → 큐 → 화면' 예제코드](#)
 - [코드 한 줄씩 살펴보기](#)
- [카운트 다운 코드 큐 기능 추가](#)
 - [큐 기능이 추가된 코드](#)

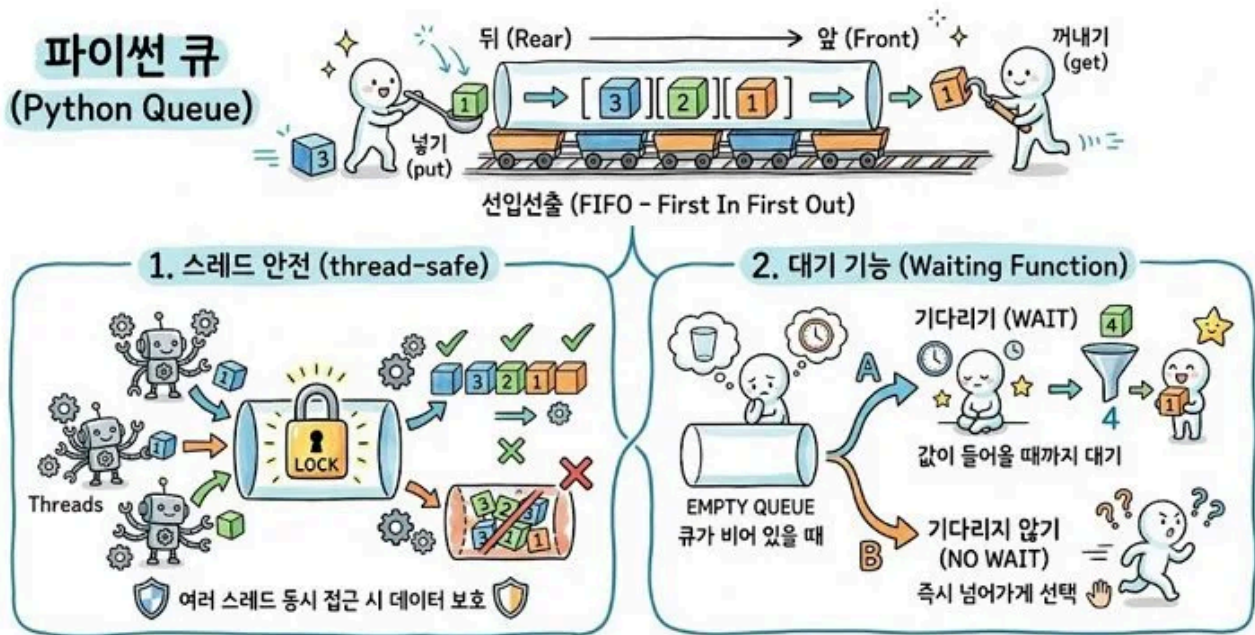
큐(Queue) 개념과 사용법

큐(Queue)란

- 큐는 먼저 넣은 것이 먼저 나오는(FIFO, First In First Out) 자료구조입니다.

넣기 (put) → [1] [2] [3] → 꺼내기 (get)
 뒤 앞

- 리스트로도 비슷하게 흉내 낼 수 있지만, 파이썬의 `queue.Queue`에는 두 가지 중요한 장점이 있습니다.
 - 스레드 안전(thread-safe): 여러 스레드가 동시에 넣고 꺼내도 데이터가 깨지지 않도록 내부적으로 잠금(lock)을 처리해 줍니다. 그래서 스레드 사이의 데이터 전달 통로로 안정맞춤입니다.
 - 대기 기능: 큐가 비어 있을 때 값이 들어올 때까지 기다리게 하거나, 기다리지 않고 즉시 넘어가게 하는 등 상황에 맞게 선택할 수 있습니다.



기본 사용법

- 큐 만들기

```
import queue
q = queue.Queue()          # 크기 제한 없는 큐
q = queue.Queue(maxsize=5) # 최대 5개까지만 담는 큐
```

- 값 꺼내기 (`get` , `get_nowait`)

```
n = q.get()
n = q.get_nowait()
```


- `get()` 은 값이 올 때까지 멈춰 서서 기다립니다. GUI 메인 스레드에서 이걸 쓰면 화면이 얼어붙으므로 주의해야 합니다.
- `get_nowait()` 는 기다리지 않으므로 GUI에 적합합니다. 대신 큐가 비어 있으면 예외가 나므로 try/except로 감싸 처리합니다.

- 상태 확인

```
q.empty()           # 비어 있으면 True
q.qsize()           # 대략적인 크기 (참고용)
```

스레드와 큐 활용하기

'스레드 → 큐 → 화면' 예제코드

- 백그라운드에서 1초마다 숫자를 세어 큐에 넣고, 메인 스레드가 이를 꺼내 라벨에 표시합니다.

```

import tkinter as tk
import threading
import queue
import time

root = tk.Tk()
label = tk.Label(root, text="대기 중", font=("", 20))
label.pack(padx=40, pady=40)

q = queue.Queue()

def work():
    for i in range(1, 10):
        time.sleep(1)
        q.put(i)

def poll():
    try:
        while True:
            n = q.get_nowait()
            label.config(text=f"{n} 초")
    except queue.Empty:
        pass
    root.after(100, poll)

threading.Thread(target=work, daemon=True).start()
root.after(100, poll)
root.mainloop()

```



코드 한 줄씩 살펴보기

- 큐 만들기

```
q = queue.Queue()
```

- 스레드 사이에 데이터를 안전하게 주고받는 통로입니다. 백그라운드는 여기에 넣고, 메인은 여기서 꺼냅니다.

- 백그라운드에서 결과 넣기 (`work`)

```
def work():
    for i in range(1, 10):
        time.sleep(1)
        q.put(i)
```

- 이 함수는 백그라운드 스레드에서 실행됩니다. 느린 작업(`time.sleep`)을 처리하고 결과를 `q.put(i)` 으로 큐에 넣기만 합니다.
- 위젯(`label`)을 전혀 건드리지 않는 것이 핵심입니다.

- 메인에서 꺼내 화면 바꾸기 (`poll`)

```
def poll():
    try:
        while True:
            n = q.get_nowait()
            label.config(text=f"{n} 초 ")
    except queue.Empty:
        pass
    root.after(100, poll)
```

- `after` 로 예약되어 메인 스레드에서 실행되므로 위젯 조작이 안전합니다.
- `get_nowait()` 는 큐가 비면 기다리지 않고 바로 예외를 냅니다. `while` 로 쌓인 값을 한 번에 처리하고, 비면 조용히 빠져나옵니다.
- 마지막으로 다시 `after(100, poll)` 로 자기 자신을 예약해, 100밀리초마다 큐를 확인하는 순환을 이룹니다.

- 스레드 시작

```
threading.Thread(target=work, daemon=True).start()
root.after(100, poll)
```

- `work` 를 백그라운드 스레드로 시작합니다. `daemon=True` 는 창이 닫히면 스레드도 함께 종료하라는 뜻입니다.
- 동시에 `poll` 을 예약해, 스레드가 큐에 넣는 결과를 메인 스레드가 꾸준히 화면에 반영합니다.

카운트 다운 코드 큐 기능 추가

- 앞서 스레드 강좌를 위한 카운트다운 예제 코드는 백그라운드 스레드에서 GUI를 조작하는 문제점이 있었습니다.
- 이를 큐 기능을 추가해 안전한 코드로 아래와 같이 개선했습니다.

큐 기능이 추가된 코드

```

import tkinter as tk
import threading
import queue
import time

class CountdownApp:
    def __init__(self, root):
        self.root = root
        self.queue = queue.Queue()
        self.label = tk.Label(root, text="10", font=("Arial", 60))
        self.label.pack(expand=True)
        tk.Button(root, text="시작", command=self.start).pack(pady=10)
        self.poll()

    def start(self):
        threading.Thread(target=self.count, daemon=True).start()

    def count(self):
        for i in range(10, -1, -1):
            self.queue.put(str(i))
            time.sleep(1)
        self.queue.put("끝!")

    def poll(self):
        try:
            while True:
                self.label.config(text=self.queue.get_nowait())
        except queue.Empty:
            pass
        self.root.after(100, self.poll)

if __name__ == "__main__":
    root = tk.Tk()
    CountdownApp(root)
    root.mainloop()

```

